

Guide de Déploiement

Ce guide détaille les étapes nécessaires pour déployer l'application sur une machine vierge.

Prérequis

Logiciels Requis

- **Docker Desktop** : Version 24.0 ou supérieure.
 - Windows : Télécharger depuis <https://www.docker.com/products/docker-desktop/>
 - Activer WSL 2 (Windows Subsystem for Linux) si demandé lors de l'installation.
- **Git** : Pour cloner le dépôt (optionnel si le code est fourni en archive).

Configuration Matérielle Recommandée

- **RAM** : 8 Go minimum (16 Go recommandé).
- **Espace disque** : 5 Go disponibles pour les images Docker.
- **Webcam** : Requête pour les fonctionnalités de reconnaissance.
- **Navigateur** : Chrome, Firefox ou Edge (version récente avec support WebRTC).

Étapes de Déploiement

Étape 1 : Récupération du Code Source

```
git clone https://github.com/Isopope/ComputerVisionPRI25-26.git
cd ComputerVisionPRI25-26
```

Ou extraire l'archive ZIP fournie :

```
unzip ComputerVisionPRI25-26.zip
cd ComputerVisionPRI25-26
```

Étape 2 : Lancement de Docker Desktop

S'assurer que Docker Desktop est démarré et fonctionnel. Vérifier avec :

```
docker --version
docker-compose --version
```

Étape 3 : Construction et Démarrage des Conteneurs

Depuis la racine du projet, exécuter :

```
docker-compose up --build
```

Cette commande :

1. Construit l'image Docker du backend (Python/FastAPI).
2. Construit l'image Docker du frontend (React compilé + Nginx).
3. Démarre les deux conteneurs en réseau.

Note : La première construction peut prendre 40 à 50 minutes selon la connexion Internet (téléchargement des dépendances Python et npm).

Étape 4 : Accès à l'Application

Une fois les conteneurs démarrés, ouvrir un navigateur et accéder à :

`http://localhost:8080`

Étape 5 : Autorisation de la Webcam

Lors du premier accès, le navigateur demandera l'autorisation d'utiliser la webcam. Cliquer sur **Auto-riser** pour activer les fonctionnalités de reconnaissance.

Commandes Utiles

Commande	Description
<code>docker-compose up -d</code>	Démarre en arrière-plan.
<code>docker-compose down</code>	Arrête et supprime les conteneurs.
<code>docker-compose logs -f</code>	Affiche les logs en temps réel.
<code>docker-compose build --no-cache</code>	Reconstruction complète sans cache.
<code>docker ps -a</code>	Liste tous les conteneurs (actifs et arrêtés).

TABLE 8.2 – Commandes Docker Compose utiles.

Dépannage

Erreur "Port 8080 already in use"

Un autre service utilise déjà le port 8080. Solutions :

- Arrêter le service conflictuel.
- Modifier le port dans `docker-compose.yml` (ex : 8081:80).

La webcam ne fonctionne pas

- Vérifier que le navigateur a l'autorisation d'accéder à la caméra.
- Utiliser HTTPS si nécessaire (certains navigateurs bloquent WebRTC en HTTP pour des raisons de sécurité).
- Tester avec `chrome://settings/content/camera` pour Chrome.

Les conteneurs ne démarrent pas

- Vérifier que Docker Desktop est bien lancé.
- Consulter les logs : `docker-compose logs`.
- S'assurer d'avoir assez d'espace disque.

Mode Développement (Optionnel)

Pour contribuer au développement sans Docker :

Backend

```
cd backend
python -m venv .venv
.venv\Scripts\activate      # Windows
pip install -r requirements.txt
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Frontend

```
cd frontend
npm install
npm run dev
```

L'application sera accessible sur `http://localhost:5173` (Vite dev server).